

Spring Boot — Auto Configuration & Core Concepts

Notes based on the uploaded lecture transcript. Focus: how Spring Boot simplifies Spring Core configuration, `SpringApplication.run()`, `@SpringBootApplication`, component scanning, auto-configuration, Spring Initializr, and the Spring Boot Starter Parent.

Source basis: the lecture explains that Spring Boot keeps the same underlying Spring Core/IOC concepts while reducing manual configuration through auto-configuration. ■filecite■turn28file0■L1-L1■

1. Spring Core vs Spring Boot

Spring Core requires more manual setup: creating the `ApplicationContext`, providing configuration metadata, using `@Configuration` and `@ComponentScan`, and then retrieving beans from the container.

Spring Boot builds on the same IoC concepts but automates much of this setup, allowing the developer to focus more directly on business logic.

- Spring Core → more manual configuration.
- Spring Boot → convention + auto-configuration + starter dependencies.
- Bean lifecycle and IoC concepts remain fundamentally the same.

2. Spring Boot Is Not Only for Web Applications

The lecture emphasizes that Spring Boot does not mean web application. Spring Boot can also be used for non-web/core applications. Web applications are created by adding appropriate web dependencies.

- Spring Boot can work with Spring MVC.
- Spring Boot is independent of Spring MVC.
- Its main role is simplifying Spring configuration and setup.

3. Spring Core Example — Manual Setup

Example used in the lecture: `OrderService` depends on `PaymentService`.

```
@Component
class PaymentService {
    public void pay() {
        System.out.println("Payment Done");
    }
}

@Component
class OrderService {
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order Placed");
    }
}
```

In Spring Core, a configuration class is created and component scanning is configured. The `ApplicationContext` is then created manually.

```
@Configuration
@ComponentScan("in.dustrikes")
```

```

class AppConfig { }

ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

OrderService order =
    context.getBean(OrderService.class);

order.placeOrder();

```

4. What Spring Boot Removes

- Manual creation of the ApplicationContext in the main flow.
- Separate configuration class for ordinary component scanning.
- Manual component-scan package configuration in the common/default case.
- Large amounts of dependency/version setup through Spring Boot starters and dependency management.

5. Spring Boot Project Creation

The lecture describes two approaches.

5.1 Create the Maven project manually

Create a Maven project and add the Spring Boot starter dependency.

5.2 Use Spring Initializr

Spring Initializr generates the project structure, Maven configuration, main application class, and selected dependencies.

- Choose Java.
- Choose Maven.
- Select a Spring Boot version.
- Choose the Java version.
- Add only the dependencies required by the project.

6. Spring Boot Starter

A Spring Boot starter acts as a convenient dependency group. Instead of manually selecting many related dependencies, the starter brings in the required transitive dependencies.

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
</dependency>

```

The transcript contrasts this with manually managing individual dependency versions.

7. Spring Boot Starter Parent

The Spring Initializr-generated Maven project contains a Spring Boot Starter Parent. The lecture explains that the parent provides dependency-management information so compatible versions can be inherited instead of repeatedly specifying versions for every Spring Boot dependency.

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>...</version>
</parent>

```

Key idea: dependency versions can be centrally managed, reducing version mismatch and manual compatibility work.

8. Main Spring Boot Application Class

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            SpringBootCoreDemoApplication.class, args
        );
    }
}
```

The lecture highlights `SpringApplication.run()` as the entry point that starts the Spring Boot application and creates the application context.

9. `SpringApplication.run()` and the IoC Container

`SpringApplication.run()` returns a `ConfigurableApplicationContext`. Since `ConfigurableApplicationContext` extends `ApplicationContext`, its result can be stored in an `ApplicationContext` reference.

```
ApplicationContext context =
    SpringApplication.run(
        SpringBootCoreDemoApplication.class, args
    );
```

The important point from the lecture is that Spring Boot starts the IoC container for you.

10. `@SpringBootApplication` — The Key Annotation

`@SpringBootApplication` is presented as a composite/parent annotation. The lecture identifies three important annotations inside it:

- `@SpringBootConfiguration`
- `@EnableAutoConfiguration`
- `@ComponentScan`

Conceptually, `@SpringBootApplication` provides the combined effect of these three annotations, avoiding the need to write all three separately.

11. `@SpringBootConfiguration`

The lecture explains `@SpringBootConfiguration` as the Spring Boot application's main configuration declaration. It is the Boot-oriented equivalent of identifying the main configuration class.

The main application class therefore acts as an important configuration entry point. Additional `@Bean` methods can also be declared in the configuration class.

12. `@ComponentScan`

`@ComponentScan` tells Spring to scan packages for component classes such as `@Component` and register their beans in the IoC container.

Spring Boot uses a convention by default: it scans from the package of the main application class and its subpackages.

- Put the main application class in a suitable root package.
- Place component classes under that package.
- Then explicit package scanning is usually unnecessary.

13. Package Convention — Very Important

The lecture describes Spring Boot as an opinionated framework: it provides defaults/conventions so less configuration is required.

```
in.dustrikes.springbootdemo
■■■ SpringBootApplication
■■■ OrderService
■■■ PaymentService
```

If OrderService and PaymentService are placed outside the default scanned package hierarchy, they will not automatically be discovered just because @Component is present.

If a different package structure is required, the lecture shows that the scan base package can be customized through the Spring Boot application annotation.

14. @EnableAutoConfiguration

This is the central concept of the lecture. The annotation tells Spring Boot to inspect the application/project setup and apply appropriate auto-configuration.

The lecture describes it conceptually as: look at the project and create/configure beans that appear appropriate for the application.

The important distinction is that this is not the same as the developer manually declaring every bean. Spring Boot has its own auto-configuration classes that can contribute beans and configuration.

15. Three Ways of Thinking About Bean Creation

- @Component-style configuration: developer marks a class as a component.
- @Bean-style configuration: developer declares a bean-producing method in a configuration class.
- Auto-configuration: Spring Boot can provide configuration/beans automatically based on the application's setup.

The lecture emphasizes that developers normally use @Component and @Bean directly, while auto-configuration is primarily an internal Spring Boot mechanism.

16. Spring Boot's Simplified Flow

```
Start application
↓
SpringApplication.run()
↓
Spring ApplicationContext / IoC container starts
↓
@SpringBootApplication is processed
↓
@SpringBootConfiguration
↓
@ComponentScan
↓
Find application components
↓
@EnableAutoConfiguration
↓
Apply suitable Boot auto-configurations
↓
Beans + dependencies are managed
↓
Application is ready
```

17. Constructor Injection Still Works

Spring Boot does not change the fundamental dependency-injection principle. The same constructor injection shown in Spring Core can be used.

```
@Component
class OrderService {
```

```

private final PaymentService paymentService;

public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}

public void placeOrder() {
    paymentService.pay();
    System.out.println("Order Placed");
}
}

```

With a single constructor, the lecture notes that explicit `@Autowired` is not necessary.

18. Getting Beans — Core Style vs Boot Style

The lecture demonstrates that `context.getBean()` works, but explicitly retrieving a bean from the IoC container is described as a Spring Core-style approach rather than the preferred Spring Boot application style.

```

OrderService order =
    context.getBean(OrderService.class);

order.placeOrder();

```

For a non-web application where a method must run at startup, the lecture indicates that Spring Boot provides another approach so the application can invoke startup logic without manually fetching the service. The detailed mechanism is deferred in the lecture.

19. Why Spring Boot Feels Easier

- Less boilerplate configuration.
- Convention-based component scanning.
- Automatic IoC container startup.
- Starter dependencies simplify dependency selection.
- Dependency management reduces manual version coordination.
- Auto-configuration supplies appropriate framework configuration.

20. Interview Questions

What is Spring Boot?

A Spring-based framework that simplifies application setup and configuration using conventions, starters, and auto-configuration.

Is Spring Boot only for web applications?

No. It can be used for non-web/core applications as well as web applications.

What does `@SpringBootApplication` contain?

The lecture identifies `@SpringBootApplication`, `@EnableAutoConfiguration`, and `@ComponentScan`.

What does `@ComponentScan` do?

It scans packages for Spring components and registers their beans.

What is the default package-scanning convention?

Scanning starts from the package containing the main application class and covers its subpackages.

What does `@EnableAutoConfiguration` do?

It enables Spring Boot's automatic configuration based on the application's setup.

What does SpringApplication.run() do?

It starts the Spring Boot application and creates/returns the application context.

Why use Spring Boot Starter dependencies?

They group related dependencies and simplify transitive dependency management.

Why is spring-boot-starter-parent useful?

It provides dependency-management information so compatible versions can be inherited.

Do Spring Boot and Spring Core use different IoC concepts?

No. Spring Boot simplifies the setup; the underlying Spring IoC concepts remain.

21. Quick Revision Sheet

- Spring Core → manual configuration.
- Spring Boot → convention + automation.
- SpringApplication.run() → starts the Boot application/context.
- @SpringBootApplication → @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan.
- @ComponentScan → finds component classes.
- Default scanning → main application package + subpackages.
- @EnableAutoConfiguration → lets Boot apply suitable auto-configuration.
- Starter → convenient dependency group.
- Starter Parent → centralized dependency management.
- Constructor injection → still works exactly as a Spring IoC concept.
- Spring Boot ≠ only web applications.

22. One-Line Mental Model

Spring Boot = Spring Core/IoC concepts + sensible conventions + starter dependencies + automatic configuration, so the developer writes much less setup code and can focus on application logic.